

eqp

eqp

Student assignment: design and build the digital eqp platform

A real-world software engineering challenge for second-year HBO students.

Four teams • Four products • One future platform

React • C# / .NET • Microsoft Azure

eqp

1. About eqp

eqp is a young IT services company that places IT professionals with different clients. We are building an organisation in which the professional is at the centre.

eqp wants to grow without automatically building a large back office. Smart software should keep administration simple, make information easy to access and give employees as much control as possible.

Today, several processes are still handled manually or in separate systems: receiving and assessing staffing requests, keeping track of skills and experience, submitting professionals for assignments and registering hours.

The ambition

eqp wants to explore how these processes could be supported by one digital platform in the future. That is where you come in.

2. The challenge

As a team, you will design one part of the future eqp platform and turn it into a working application.

Product team	Focus
1. Opportunity Hub	Requests and opportunities
2. eqp Skills	Skills, experience and professional profiles
3. My eqp	Hours, activities and leave
4. eqp Flow	Submissions, deployment and feedback

One important extra challenge

The four applications should be able to become part of one integrated eqp platform in the future. Each team builds a useful standalone product — but not an isolated island.

3. Technical principles

The frontend will be built in React. For the backend, we would like you to use C# and .NET. The application must be designed so it can run on the Microsoft Azure platform.

We deliberately do not prescribe the complete technical architecture. A key part of the assignment is researching, choosing and defending an appropriate solution.

- React frontend structure
- .NET backend structure
- REST APIs or other communication patterns
- databases and data storage
- authentication and authorization

eqp

- integration with external services
- hosting on Azure
- secrets and configuration
- logging and monitoring
- testing and CI/CD
- scalability
- security and privacy

Do not start coding immediately

Work roughly in this order: Problem → users → requirements/user stories → UX → domain model → architecture → API → implementation → testing → deployment → demonstration.

4. Shared architecture

The four teams work on different products, but those products should eventually be able to exchange data. An employee in eqp Skills, for example, must be the same employee who records hours in My eqp or is submitted for an opportunity in eqp Flow.

- Employee
- Skill
- Certification
- Opportunity
- Submission
- Assignment
- Customer
- Activity

During regular architecture syncs, representatives from all teams discuss shared domain objects, data ownership, identification, APIs, authentication and authorization, and how to keep dependencies between teams small.

Integration

Full integration of all four applications is not required. It must, however, be credible that integration could be added later.

5. Team 1 – Opportunity Hub

The situation

eqp receives requests for IT professionals, for example a .NET developer, DevOps engineer, tester or data engineer. A request may contain a client, role, required skills, location, hours, start date, duration, rate and response deadline. Some of these requests arrive by email.

Your assignment

Design and build an application that allows the eqp back office to manage incoming requests.

eqp

- view and register opportunities
- capture relevant attributes
- search and filter
- change status
- see which opportunities need attention
- design a logical status model

Gmail and test data

eqp uses Gmail. Explore how requests could be retrieved from a mailbox, including authentication, security and privacy. The first version may use simulated emails. eqp will provide approximately 5–10 sample emails.

Architecture question

How do you make Gmail just one possible input source, so another mailbox, portal or API could be added later?

Stretch goal

Can software automatically recognise role, skills, location, start date, rate and other relevant information in an email? AI is allowed, but not required.

6. Team 2 – eqp Skills

The situation

A traditional CV is a snapshot, while skills and experience keep changing. Employees work on new projects, learn technologies and gain certifications. eqp therefore wants a living professional profile.

Your assignment

Design and build an application in which employees can maintain their professional profile and the back office can search those profiles.

- programming languages, frameworks, tools and platforms
- certifications
- projects and work experience
- experience level or years of experience
- education
- personal preferences or ambitions
- searching and filtering by the back office

Test data

eqp will provide one or two fictional CVs that can be used as starting data and test data.

Architecture question

Is .NET one skill? How does it relate to C#, ASP.NET Core, Entity Framework and Azure? How do you represent experience level, and who is allowed to determine it?

eqp

Stretch goal: matching

Design a first, explainable version of Opportunity → required skills → available employees → possible match. A complex AI model is not necessary.

7. Team 3 – My eqp

The situation

eqp employees usually work at client organisations. Administration should be as simple as possible for them, while eqp still needs information such as hours worked and leave.

Your assignment

Design and build a mobile-first React application for an employee's day-to-day administration.

- register and edit hours
- record activities
- view previous registrations
- request leave
- view leave request status
- design a personal dashboard

External systems

eqp uses an external system for HR administration. Not all information therefore needs to be owned by your application.

Architecture question

Which application owns which data? What is the source of truth? Design the solution so an external HR system can be connected later without rebuilding the whole application.

Stretch goal: financial insight

A future version could show an employee revenue, costs and the build-up of a bonus or profit share. Explore how such privacy-sensitive information could be shown securely.

8. Team 4 – eqp Flow

The situation

Finding the right assignment is a process: available → opportunity → submitted → interview → placed or rejected. eqp does not only want to record the outcome, but also learn why a submission succeeds or fails.

Your assignment

Design and build an application that lets eqp follow the process from availability to placement.

eqp

- show available professionals
- link a professional to an opportunity
- register a submission
- track status
- capture feedback and rejection reasons
- show a timeline of events
- design at least one useful process dashboard

Possible rejection reasons

Examples include insufficient experience, missing technical skills, rate, availability, location, another candidate selected or unknown reason.

Product question

Which statuses, feedback and KPIs actually help eqp make better decisions?

Possible insights

For example: number of submissions, interviews and placements, average lead time and recurring rejection reasons.

9. Authentication and roles

Not every user may see or change the same information. At minimum, consider the roles Employee and Back office/administrator. An employee may edit their own profile, for example, but not simply edit a colleague's.

- authentication
- authorization
- roles and permissions
- personal data
- API security
- secure data storage

10. API-first

Do not build your application as an isolated island. The frontend should communicate with the underlying functionality through a clear application layer/API.

A possible structure is: React frontend → .NET API → business logic → data. Where needed, the .NET API communicates with external services such as Gmail or an HR system.

- Document your API so another team can understand how to use it.

11. MVP versus ambition

We do not expect a complete production platform. A smaller application that is technically sound and genuinely useful is more valuable than a large amount of half-finished functionality.

eqp

- Must have – required to solve the core problem
- Should have – valuable, but not required for the first version
- Could have – interesting extension if time allows

12. Deliverables

- 1. Problem analysis** — Who is the user, what problem are you solving and why does it matter?
- 2. Requirements and user stories** — Including MVP prioritisation.
- 3. UX/UI design** — From early concepts/wireframes to the final interface.
- 4. Domain model** — Key objects, relationships and responsibilities.
- 5. Software architecture** — Components, interfaces, databases and external systems, including rationale.
- 6. API documentation** — So other teams understand how your system can communicate.
- 7. Working software** — React frontend with C#/.NET backend and the required functionality.
- 8. Test strategy and tests** — What do you test, and why?
- 9. Azure & deployment** — Demonstrably suitable for Microsoft Azure. Explain your chosen Azure components. A working Azure deployment is preferred.
- 10. Technical documentation** — A new developer should be able to understand, run locally, deploy and continue the project.

13. Final presentation

At the end, each team presents its product to eqp. This is explicitly not a PowerPoint presentation about what you did.

Show us the product.

Show the problem you solved, how a user actually uses the product, the choices you made, the architecture, how the solution can run on Azure, what works, what does not work yet, and what you would build with three more months.

Also answer one central question:

Could eqp make your application part of one larger eqp platform tomorrow?

14. What will we look at?

We are not only assessing how much functionality you built. A good solution shows that you can make and defend choices as software engineers.

- understanding of the problem
- quality of the user experience
- code quality and maintainability
- software architecture and rationale

eqp

- good use of C#/.NET and appropriate use of Azure
- security and privacy
- API design and testability
- collaboration with other teams
- quality of the working product
- critical reflection on your own result

Our principle

A technically complicated solution is not automatically a good solution. The best solution solves the problem well, is understandable and can grow.

15. The shared final challenge

Opportunity Hub knows which opportunities are available. eqp Skills knows the skills and experience of employees. My eqp supports employees in their day-to-day administration. eqp Flow knows which professional was submitted for which opportunity and what the result was.

Each product has value on its own. Together, they become much more interesting.

A future eqp platform could, for example, know:

“A new request for a senior .NET developer has just arrived. Which available employee is a good fit, what relevant experience do they have, and what have we learned from previous submissions?”

You do not have to build that entire platform. Your assignment is to make sure that what you build today could become part of that platform tomorrow.